Computer Vision Course

School of Information and Communication Technology

1

ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

Computer Vision

Chapter 6: Energy-based and Scored-based models

Computer Vision Group
School of Information and Communications Technology

ONE LOVE. ONE FUTURE.

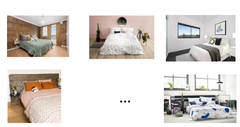
2

Contents

- Energy-based models (EBM)
- Score-based models (SBM)

Statistical Generative Models

Statistical generative models are **learned from data**

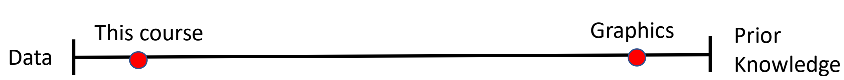


Data
(e.g., images of bedrooms)

+

Prior Knowledge
(e.g., physics, materials, ..)

Priors are always necessary, but there is a spectrum



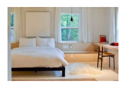
Statistical Generative Models

A statistical generative model is a **probability distribution** p(x)

- **Data:** samples (e.g., images of bedrooms)
- **Prior knowledge:** parametric form (e.g., Gaussian?), loss function (e.g., maximum likelihood?), optimization algorithm, etc.

Image x  → A probability distribution p(x) → scalar probability p(x)

It is generative because **sampling from p(x) generates new images**

 ... 

Building a simulator for the data generating process

Data simulator → Data

Building a simulator for the data generating process

Control signals → Data simulator → New datapoints

ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

7

Building a simulator for the data generating process

Control signals → Data simulator → New datapoints

Potential datapoints → Data simulator → Probability values

ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

8

Building a simulator for the data generating process

Control signals → Data simulator =Statistical model → New datapoints

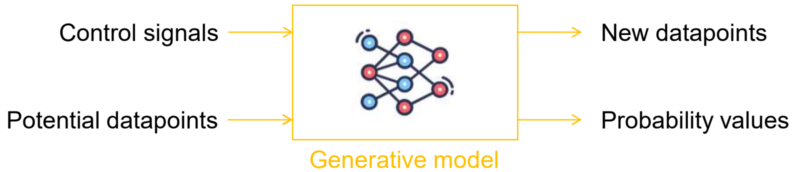
Potential datapoints → Data simulator =Statistical model → Probability values

Building a simulator for the data generating process

Control signals → Data simulator =Statistical model =Generative model → New datapoints

Potential datapoints → Data simulator =Statistical model =Generative model → Probability values

Building a simulator for the data generating process



Parameterizing probability distributions

- Probability distributions p(x) are a key building block in generative modeling.
  - non-negative: p(x) ≥ 0
  - sum-to-one: $\sum_x p(x) = 1$ (or $\int p(x)dx = 1$ for continuous variables)
- Coming up with a non-negative function $p_\theta(x)$ is not hard but sum-to-one is key
- Problem: $g_\theta(x) \geq 0$ is easy, but $g_\theta(x)$ might not be normalized
- Solution:

$$p_\theta(\mathbf{x}) = \frac{1}{Z(\theta)} g_\theta(\mathbf{x}) = \frac{1}{\int g_\theta(\mathbf{x})\mathrm{d}\mathbf{x}} g_\theta(\mathbf{x}) = \frac{1}{Volume(g_\theta)} g_\theta(\mathbf{x})$$

Parameterizing probability distributions

- Function forms $g_\theta(x)$ need to allow analytical integration.
  - Very useful as building blocks for more complex distributions
- Selected function:

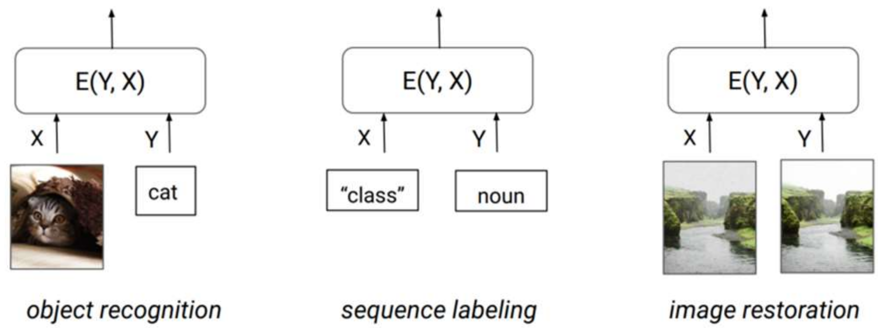
  giong function slide duoi

- Why exponential:
  - Want to capture very large variations in probability. log-probability is the natural scale we want to work with. Otherwise need highly non-smooth f_θ.
  - Exponential families. Many common distributions can be written in this form.
  - These distributions arise under fairly general assumptions in statistical physics (maximum entropy, second law of thermodynamics).
    - $-f_\theta(x)$ is called the energy, hence the name.
    - Intuitively, configurations x with low energy (high $f_\theta(x)$) are more likely

Energy-based model

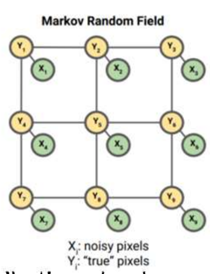
- Function: $$p_\theta(\mathbf{x}) = \frac{1}{\int \exp(f_\theta(\mathbf{x}))\mathrm{d}\mathbf{x}} \exp(f_\theta(\mathbf{x})) = \frac{1}{Z(\theta)} \exp(f_\theta(\mathbf{x}))$$
- Pros:
  - Extreme flexibility: can use pretty much any function fθ(x) you want
- Cons:
  - Sampling from $p_\theta(x)$ is hard
  - Evaluating and optimizing likelihood $p_\theta(x)$ is hard (learning is hard)
  - No feature learning (but can add latent variables)
  - The fundamental issue is that computing Z(θ) numerically (when no analytic solution is available) scales exponentially in the number of dimensions of x

Applications of Energy-based models



Example: Ising Model

- There is a true image $y \in \{0, 1\}^{3\times3}$, and a corrupted image $x \in \{0, 1\}^{3\times3}$. We know x, and want to somehow recover y.



- We model the joint probability distribution p(y, x) as

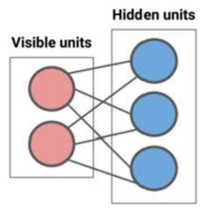
$$p(\mathbf{y}, \mathbf{x}) = \frac{1}{Z} \exp\left(\sum_i \psi_i(x_i, y_i) + \sum_{(i,j)\in E} \psi_{ij}(y_i, y_j)\right)$$

- $\psi_i(x_i, y_i)$: the i-th corrupted pixel depends on the i-th original pixel
- $\psi_{ij}(y_i, y_j)$: neighboring pixels tend to have the same value
- How did the original image y look like? Solution: maximize p(y|x). Or equivalently, maximize p(y, x).

Example: Restricted Boltzmann machine (RBM)

- RBM: energy-based model with latent variables
- Two types of variables:
  - $x \in \{0, 1\}^n$ are visible variables (e.g., pixel values)
  - $z \in \{0, 1\}^m$ are latent ones
- The joint distribution is

$$p_{W,b,c}(\mathbf{x}, \mathbf{z}) = \frac{1}{Z} \exp\left(\mathbf{x}^T W \mathbf{z} + b\mathbf{x} + c\mathbf{z}\right) = \frac{1}{Z} \exp\left(\sum_{i=1}^{n}\sum_{j=1}^{m} x_i z_j w_{ij} + b\mathbf{x} + c\mathbf{z}\right)$$



- Restricted because there are no visible-visible and hidden-hidden connections, i.e., $x_i x_j$ or $z_i z_j$ terms in the objective

Training intuition

- **Goal**: maximize $\exp\{f_\theta(x_{train})\}/Z(\theta)$
- **Intuition**: because the model is not normalized, increasing the un-normalized log-probability $f_\theta(x_{train})$ by changing θ does not guarantee that x_{train} becomes relatively more likely (compared to the rest).
- **Idea**: Instead of evaluating $Z(\theta)$ exactly, use a Monte Carlo estimate
- **Contrastive divergence algorithm**: sample $x_{sample} \sim p_\theta$, take step on $\nabla_\theta (f_\theta(\text{xtrain}) - f_\theta(\text{xsample}))$. Make training data more likely than typical sample from the model.

Contrastive Divergence

- Maximize log-likelihood: $\max_\theta f_\theta(x_{train}) - \log Z(\theta)$
- Gradient of log-likelihood:

$$\nabla_\theta f_\theta(x_{train}) - \nabla_\theta \log Z(\theta) \approx \nabla_\theta f_\theta(x_{train}) - \nabla_\theta f_\theta(x_{sample})$$

where $x_{sample} \sim \exp\{f_\theta(x_{sample})\}/Z(\theta)$

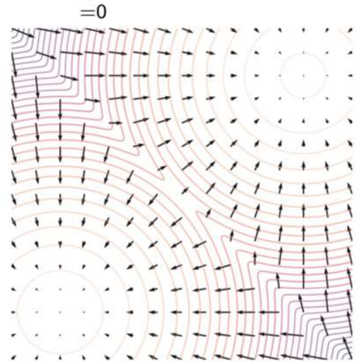
- How to sample?
  - Main issue: cannot easily compute how likely each possible sample is
  - However, we can easily compare two samples x, x′
- Use an iterative approach called Markov Chain Monte Carlo
- Works in theory, but can take a very long time to converge → Langevin MCMC

Score function

Energy-based model: $p_\theta(\mathbf{x}) = \frac{\exp\{f_\theta(\mathbf{x})\}}{Z(\theta)}$, $\log p_\theta(\mathbf{x}) = f_\theta(\mathbf{x}) - \log Z(\theta)$
(Stein) Score function:

$$s_\theta(\mathbf{x}) := \nabla_\mathbf{x} \log p_\theta(\mathbf{x}) = \nabla_\mathbf{x} f_\theta(\mathbf{x}) - \underbrace{\nabla_\mathbf{x} \log Z(\theta)}_{=0} = \nabla_\mathbf{x} f_\theta(\mathbf{x})$$

- Gaussian distribution
  $p_\theta(x) = \frac{1}{\sqrt{2\pi\sigma}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$
  $\longrightarrow s_\theta(x) = -\frac{x-\mu}{\sigma^2}$
- Gamma distribution
  $p_\theta(x) = \frac{\beta^\alpha}{\Gamma(\alpha)} x^{\alpha-1} e^{-\beta x}$
  $\longrightarrow s_\theta(x) = \frac{\alpha-1}{x} - \beta$



$p_\theta(\mathbf{x})$ vs. $s_\theta(\mathbf{x})$

Score matching

- Observation
  - sθ(x) = ∇x log pθ(x) is independent of the partition function Z(θ).
- Fisher divergence between p(x) and q(x):

$$D_F(p, q) := \frac{1}{2} E_{\mathbf{x} \sim p}[\|\nabla_{\mathbf{x}} \log p(\mathbf{x}) - \nabla_{\mathbf{x}} \log q(\mathbf{x})\|_2^2]$$

- Score matching: minimizing the Fisher divergence between pdata(x) and the EBM pθ(x) ∝ exp{fθ(x)}

$$\frac{1}{2} E_{\mathbf{x} \sim p_{\text{data}}}[\|\nabla_{\mathbf{x}} \log p_{\text{data}}(\mathbf{x}) - s_\theta(\mathbf{x})\|_2^2]$$

$$= \frac{1}{2} E_{\mathbf{x} \sim p_{\text{data}}}[\|\nabla_{\mathbf{x}} \log p_{\text{data}}(\mathbf{x}) - \nabla_{\mathbf{x}} f_\theta(\mathbf{x})\|_2^2]$$

Score matching

- Sample a mini-batch of datapoints $\{x_1, x_2, \cdots, x_n\} \sim p_{data}(x)$
- Estimate the score matching loss with the empirical mean

$$\frac{1}{n} \sum_{i=1}^{n} \left[\frac{1}{2} \|\nabla_{\mathbf{x}} \log p_\theta(\mathbf{x}_i)\|_2^2 + \text{tr}(\nabla_{\mathbf{x}}^2 \log p_\theta(\mathbf{x}_i)) \right]$$

$$= \frac{1}{n} \sum_{i=1}^{n} \left[\frac{1}{2} \|\nabla_{\mathbf{x}} f_\theta(\mathbf{x}_i)\|_2^2 + \text{trace}(\nabla_{\mathbf{x}}^2 f_\theta(\mathbf{x}_i)) \right]$$

- Stochastic gradient descent.
- No need to sample from the EBM!

→ Denoising score matching (Vincent 2010) and sliced score matching (Song et al. 2019).

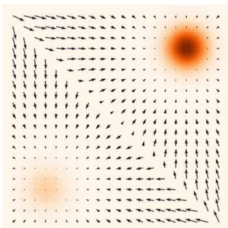
Other approach for sampling-free training

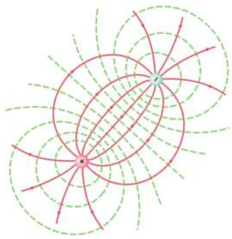
- Noise contrastive estimation
  - Additionally provides an estimate of the partition function.
- Adversarial optimization

How to represent probability distributions?

- When the probability distribution function is differentiable, we can compute the gradient of a probability density.

Score function $\nabla_{\mathbf{x}} \log p(\mathbf{x})$

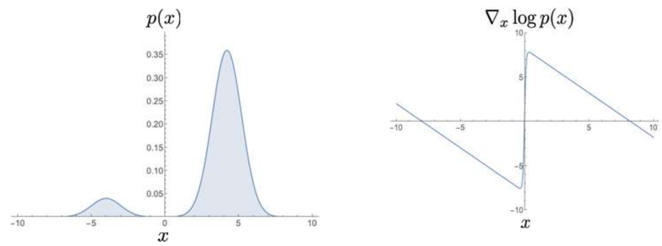

(pdf and score)


(Electrical potentials and fields)

How to represent probability distributions?

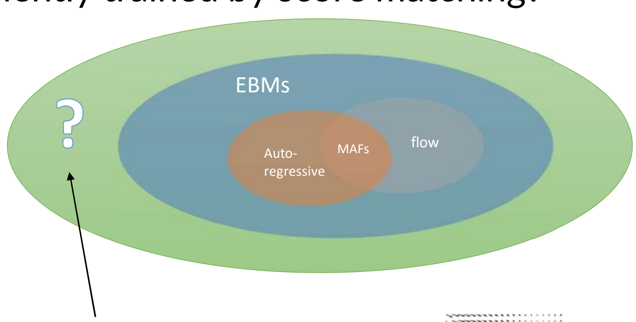
- When the probability distribution function is differentiable, we can compute the gradient of a probability density.

Score function $\nabla_{\mathbf{x}} \log p(\mathbf{x})$



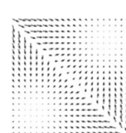
Score-based models

- What's the most general model that can be efficiently trained by score matching?



- Score-based model

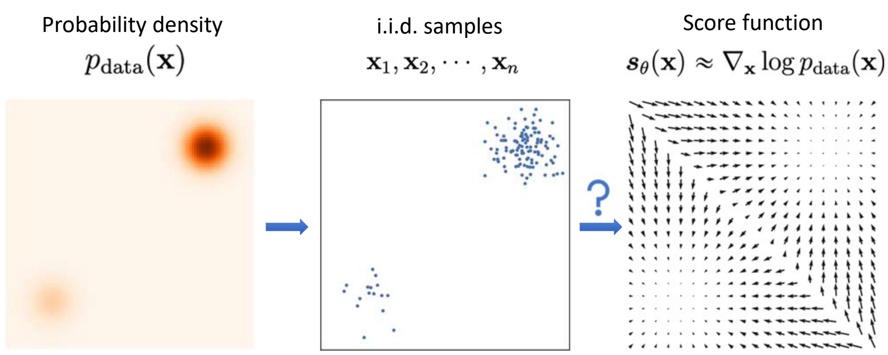
$s_\theta(\mathbf{x})$



$\mathbb{R}^d \to \mathbb{R}^d$

Directly model the vector field of gradients!

Score estimation by training score-based models

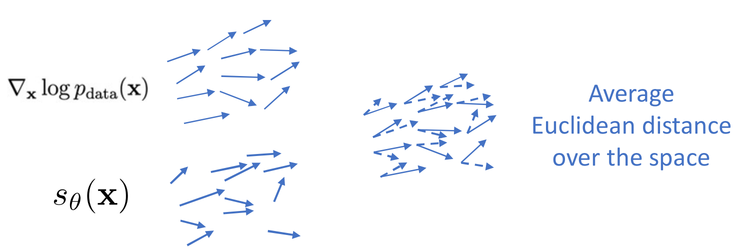


ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

29

Score estimation by training score-based models

- **Given:** i.i.d. samples $\{\mathbf{x}_1, \mathbf{x}_2, \cdots, \mathbf{x}_n\} \sim p_{\text{data}}(\mathbf{x})$
- **Task:** Estimating the score $\nabla_\mathbf{x} \log p_{\text{data}}(\mathbf{x})$
- **Score Model:** A learnable vector-valued function $\mathbf{s}_\theta(\mathbf{x}) : \mathbb{R}^d \to \mathbb{R}^d$
- **Goal:** $\mathbf{s}_\theta(\mathbf{x}) \approx \nabla_\mathbf{x} \log p_{\text{data}}(\mathbf{x})$
- How to compare two vector fields of scores?



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

30

Score estimation by training score-based models

- **Objective:** Average Euclidean distance over the whole space.

$$\frac{1}{2} E_{\mathbf{x} \sim p_{\text{data}}}[\|\nabla_{\mathbf{x}} \log p_{\text{data}}(\mathbf{x}) - \boldsymbol{s}_\theta(\mathbf{x})\|_2^2]$$

(Fisher divergence)

- **Score matching:**

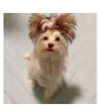
$$E_{\mathbf{x} \sim p_{\text{data}}}\left[\frac{1}{2}\|\boldsymbol{s}_\theta(\mathbf{x})\|_2^2 + \operatorname{tr}(\underbrace{\nabla_{\mathbf{x}} \boldsymbol{s}_\theta(\mathbf{x})}_{\text{Jacobian of } \boldsymbol{s}_\theta(\mathbf{x})})\right]$$

- **Requirements:**
  - The score model must be efficient to evaluate.
  - Do we need the score model to be a proper score function (i.e., gradient of a scalar "energy" function)?

ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

31

Denoising Score Matching (Vincent, 2011)



$p_{\text{data}}(\mathbf{x})$

$\mathbf{x}$

$q_\sigma(\tilde{\mathbf{x}} \mid \mathbf{x})$

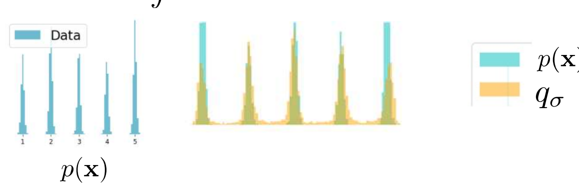
$q_\sigma(\tilde{\mathbf{x}})$

$\tilde{\mathbf{x}}$

- Consider the perturbed distribution

$$q_\sigma(\tilde{\mathbf{x}} \mid \mathbf{x}) = \mathcal{N}(\mathbf{x}; \sigma^2 I)$$

$$q_\sigma(\tilde{\mathbf{x}}) = \int p(\mathbf{x}) q_\sigma(\tilde{\mathbf{x}} \mid \mathbf{x}) \mathrm{d}\mathbf{x}$$



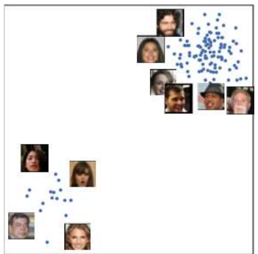
- Score estimation for $\nabla_{\tilde{\mathbf{x}}} \log q_\sigma(\tilde{\mathbf{x}})$ is easier
- If the noise level is small, this is a good approximation

$$q_\sigma(\tilde{\mathbf{x}}) \approx p(\tilde{\mathbf{x}})$$

ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

32

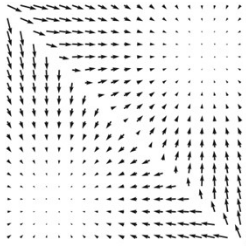
Score-based generative modeling



Data samples

$\{\mathbf{x}_1, \mathbf{x}_2, \cdots, \mathbf{x}_n\} \sim p_{\text{data}}(\mathbf{x})$

Score Matching



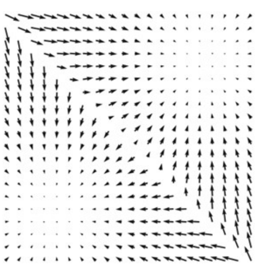
Scores

$\boldsymbol{s}_\theta(\mathbf{x}) \approx \nabla_\mathbf{x} \log p_{\text{data}}(\mathbf{x})$

?

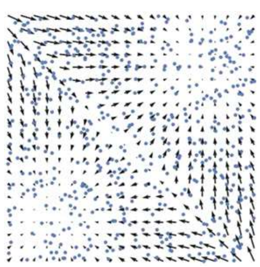
New samples

From scores to samples: Langevin MCMC



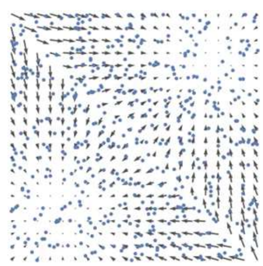
Scores

$\mathbf{s}_\theta(\mathbf{x})$



Follow the scores

$\tilde{\mathbf{x}}_{t+1} \leftarrow \tilde{\mathbf{x}}_t + \frac{\epsilon}{2}\mathbf{s}_\theta(\tilde{\mathbf{x}}_t)$



Follow noisy scores: Langevin MCMC

$\mathbf{z}_t \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$

$\tilde{\mathbf{x}}_{t+1} \leftarrow \tilde{\mathbf{x}}_t + \frac{\epsilon}{2}\mathbf{s}_\theta(\tilde{\mathbf{x}}_t) + \sqrt{\epsilon}\,\mathbf{z}_t$

Langevin dynamics sampling

- Sample from $p(\mathbf{x})$ using only the score $\nabla_{\mathbf{x}} \log p(\mathbf{x})$
- Initialize $\mathbf{x}^0 \sim \pi(\mathbf{x})$
- Repeat for $t \leftarrow 1, 2, \cdots, T$

$$\mathbf{z}^t \sim \mathcal{N}(\mathbf{0}, \boldsymbol{I})$$
$$\mathbf{x}^t \leftarrow \mathbf{x}^{t-1} + \frac{\epsilon}{2}\nabla_{\mathbf{x}} \log p(\mathbf{x}^{t-1}) + \sqrt{\epsilon}\, \mathbf{z}^t$$

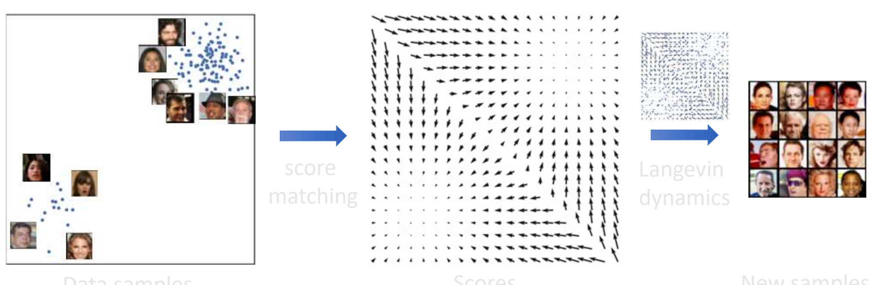
- If $\epsilon \to 0$ and $T \to \infty$, we are guaranteed to have

$$\mathbf{x}^T \sim p(\mathbf{x})$$

- Langevin dynamics + score estimation

$$\boldsymbol{s}_\theta(\mathbf{x}) \approx \nabla_{\mathbf{x}} \log p(\mathbf{x})$$

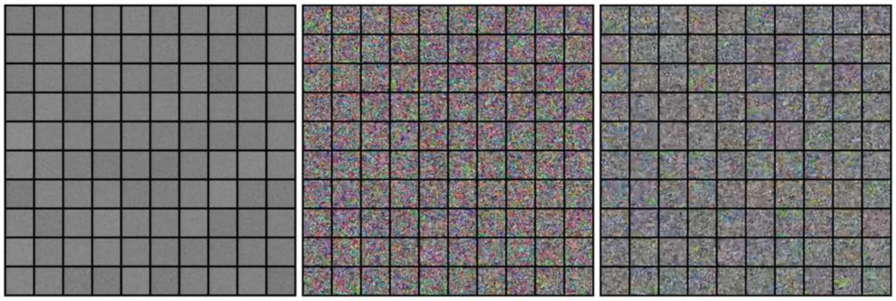
Score-based generative modeling



$\{\mathbf{x}_1, \mathbf{x}_2, \cdots, \mathbf{x}_n\} \sim p_{\text{data}}(\mathbf{x})$

$\boldsymbol{s}_\theta(\mathbf{x}) \approx \nabla_{\mathbf{x}} \log p_{\text{data}}(\mathbf{x})$

Score-based generative modeling: results

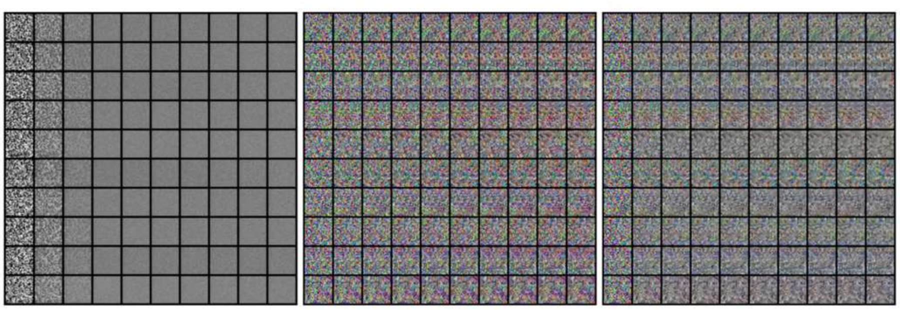


(a) MNIST (b) CelebA (c) CIFAR-10

Final samples

37

Score-based generative modeling: results



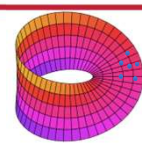
(a) MNIST (b) CelebA (c) CIFAR-10

Langevin sampling process

38

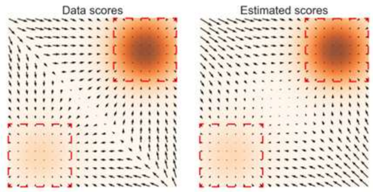
Pitfalls

- Manifold hypothesis. Data score is undefined.

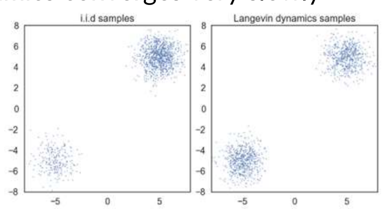


$$\times \nabla_{\mathbf{x}} \log p_{\text{data}}(\mathbf{x})$$

- Score matching fails in low data density regions

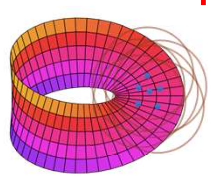


- Langevin dynamics converges very slowly

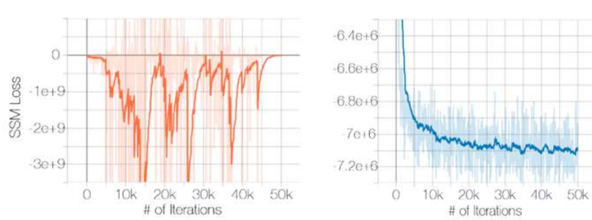


Gaussian perturbation

- The solution to all pitfalls: **Gaussian perturbation!**
- Manifold + noise



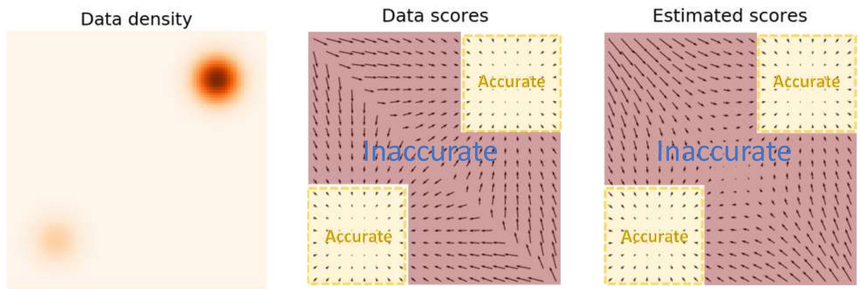
- Score matching on noisy data.



CIFAR-10 Noisy CIFAR-10

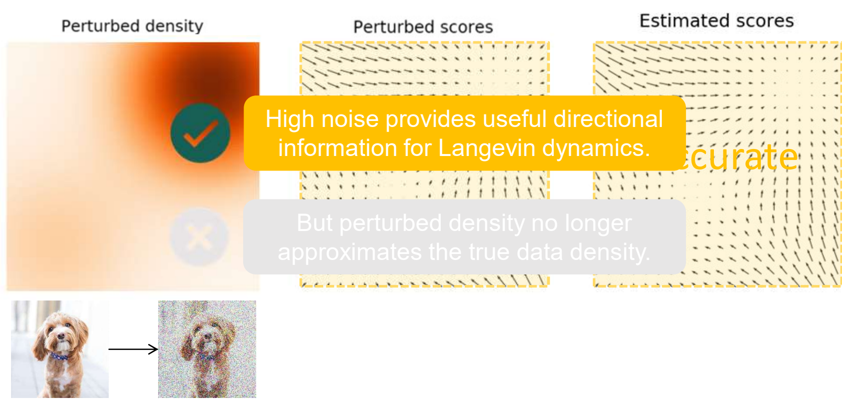
$\mathcal{N}(0; 0.0001)$

Challenge in low data density regions



Song and Ermon. "Generative Modeling by Estimating Gradients of the Data Distribution." NeurIPS 2019.

Improving score estimation by adding noise



High noise provides useful directional information for Langevin dynamics.

But perturbed density no longer approximates the true data density.

Multi-scale Noise Perturbation

- How much noise to add?

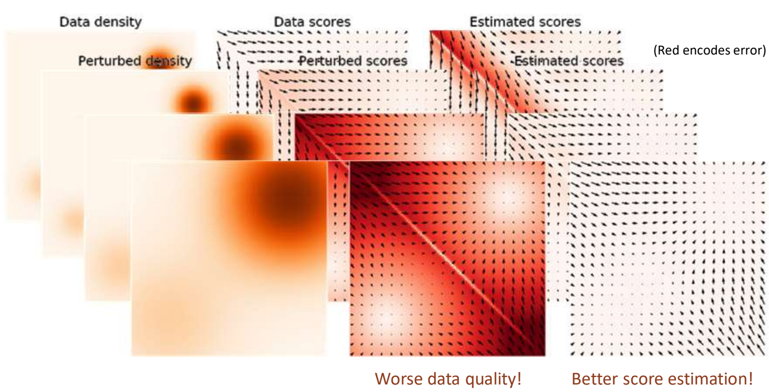
- Multi-scale noise perturbations.

$$\sigma_1 > \sigma_2 > \cdots > \sigma_{L-1} > \sigma_L$$

ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

48

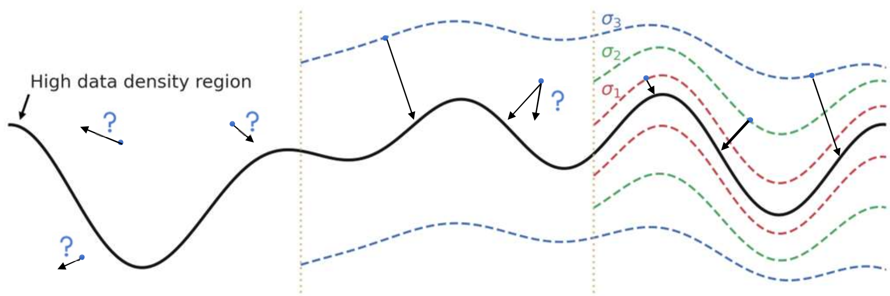
Trading off Data Quality and Estimation Accuracy



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

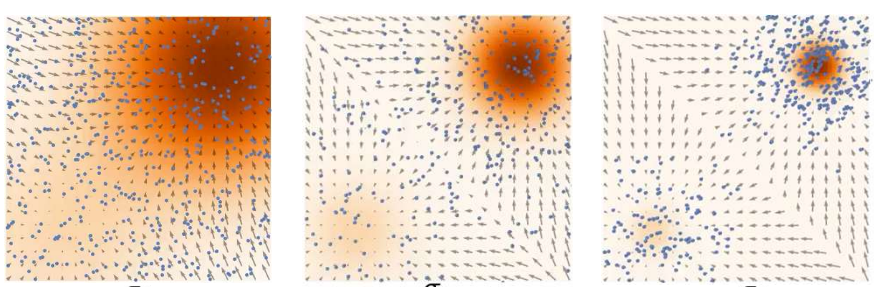
49

Using multiple noise scales



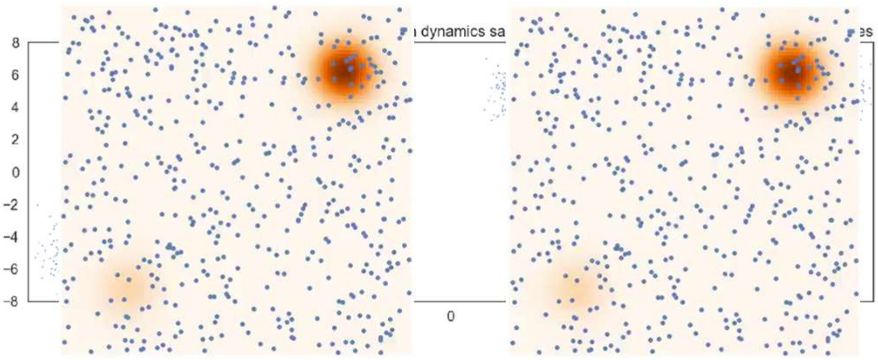
Annealed Langevin Dynamics: Joint Scores to Samples

- Sample using $\sigma_1, \sigma_2, \cdots, \sigma_L$ sequentially with Langevin dynamics.
- Anneal down the noise level.
- Samples used as initialization for the next level.



σ_1 σ_2 σ_3

Comparison to the vanilla Langevin dynamics



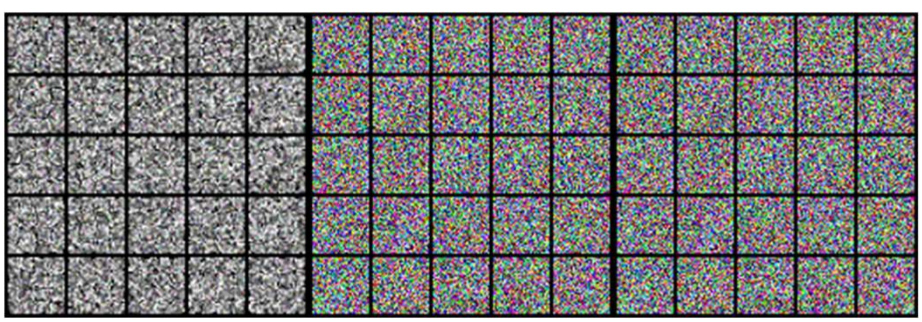
Langevin dynamics

Annealed Langevin dynamics

ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

52

Experiments: Sampling



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

53

THANK YOU !

54